# AnubisEngine Study Guide

## A practical map of the engine, its Vulkan path, and how to relearn it

Prepared for the project author | Source reviewed 5 October 2026

**Purpose** Use this document to rebuild a mental model of your code after time away. Read it in short sessions, then open the referenced source and explain the flow in your own words.

### How to use this guide

Start with the one-page mental model, then follow a concrete path through startup, a frame, and a draw. The later sections describe each subsystem, the asset/config workflow, practical source-reading routes, and recall exercises. Treat code observations as a snapshot: architecture and behavior can change as you work.

This describes the repository contents inspected for this guide. It does not claim that the current checkout was built or run during authoring.

## Contents


- 1. One-page mental model
- 2. Repository map
- 3. Startup and ownership
- 4. Frame loop and Vulkan rendering
- 5. Scene, model, and animation flow
- 6. Renderer architecture
- 7. Core services and math
- 8. Assets and configuration
- 9. Build and runtime notes
- 10. Known code facts to recheck
- 11. Study plan and self-quiz
- 12. Glossary and source index


## 1. One-page mental model

AnubisEngine is a modular C++20 engine with a shared rendering interface and two backends. The sample application creates CAnubisEngine; the engine registers services, opens a GLFW window, creates the chosen render device, loads assets and pipelines, and initializes a scene. Each tick processes events and input, updates the scene, and asks the render device to record and present a frame.

| Question | Short answer |
|---|---|
| Where does execution begin? | Sandbox/Main.cpp |
| Who coordinates engine lifecycle and frame order? | CAnubisEngine in Engine/AnubisEngine.cpp |
| What keeps renderer-independent API types? | Engine/Core/include/API |
| Where does Vulkan backend logic live? | Engine/Renderer/include/Vulkan, src/Vulkan, and Device/VulkanRenderDevice.* |
| How do scene objects reach draw calls? | Components build SRenderInstance records; CRenderQueue gathers/sorts them; a backend binds resources and issues draws. |
| Where are startup asset choices | Assets/Config JSON files, interpreted by managers and renderer setup. |

| | |
|---|---|
| configured? | |

**Remember** Follow one object and one frame end to end. That is more memorable than trying to memorize every class name.

# 2. Repository map

| Path | Responsibility | Good entry point |
|---|---|---|
| Sandbox/ | Small executable that initializes the engine and runs the loop. | Main.cpp |
| Engine/ | Engine facade plus subsystem libraries. | AnubisEngine.h/.cpp |
| Engine/Core/ | Render API contracts and common services: window, input, time, events, config, logging, memory. | include/API/RenderDevice.h |
| Engine/Renderer/ | OpenGL and Vulkan devices, resources, pipelines, descriptors, command lists, camera, textures. | src/Device/VulkanRenderDevice.cpp |
| Engine/Model/ | Meshes, actors, importing, animation, skin palettes, render queue. | src/Services/ActorsManager.cpp |
| Engine/Scene/ | Scene/entity/component representation and update/render traversal. | src/Core/SceneManager.cpp |
| Engine/Math/ | Vectors, matrices, quaternions, transforms, rays, frusta, bounds, projections. | include/EngineMath.h |
| Engine/Terrain/ | Terrain and terrain patch types. | include/Terrain/Terrain.h |
| Assets/ | Runtime JSON config and model, animation, character texture and related assets. | Config/EngineActors.json |
| shaders/ | Backend shader sources; Vulkan directory also contains SPIR-V artifacts and compile.bat. | vulkan/static_mesh_shader.vert |
| Extern/ | Third-party headers and prebuilt dependencies. | Usually skip during first reread. |

The root CMake file adds bundled GLAD and stb_image, then Engine and Sandbox. The Editor target is optional and disabled by default in the root configuration; the current tree listing did not show an Editor directory, so do not assume that target is ready.

# 3. Startup and ownership

## The application entry

Sandbox/Main.cpp constructs CAnubisEngine, initializes it with API_OPENGL, then repeatedly updates CTimerManager, checks CWindowManager::ShouldClose, and calls engine.Tick(dt). It calls engine.Destroy at exit. CAnubisEngine::Run contains a similar loop, but Sandbox currently owns the loop itself.

| Startup stage | What happens | Source to inspect |
|---|---|---|
| Choose API | The caller selects OpenGL or Vulkan. CreateRenderDevice constructs the matching implementation. | Sandbox/Main.cpp; AnubisEngine::CreateRenderDevice |
| Register services | Managers and the event bus become reachable through CServiceLocator. | AnubisEngine::Initialize; ServiceLocator.h |
| Initialize core | Logging, config, timer, window and input are initialized. | AnubisEngine::Initialize |
| Initialize rendering | The render device receives the GLFW window handle and creates backend state/resources. | CIRenderDevice; backend Initialize |
| Load and scene | Render setup loads actors/pipelines; SceneManager creates the initial scene/entity when data is available. | VulkanRenderDevice::Initialize; SceneManager::Initialize |
| Events | Callbacks/events update input state, resize the renderer, and handle window close. | AnubisEngine::InitializeEvents |

## Service locator

CServiceLocator stores pointers to services and allows modules to retrieve them without passing every manager through every call. The AnubisInstance / AnubisInstancePtr / AnubisHasService macros are convenience wrappers. This makes

dependencies easy to access but less visible at call sites; when reading a function, search for CServiceLocator::Get to discover its real dependencies.

### Shutdown

CAnubisEngine::Destroy clears the scene, shuts down and deletes the render device, destroys the window/input/timer, dumps memory leak information, destroys config/logging, then clears service registrations. Backend Shutdown must release GPU objects before the Vulkan device is destroyed. Follow the exact order when changing ownership or resource lifetime.

# 4. Frame loop and Vulkan rendering

### Engine-level tick

1. HandleInput polls GLFW events, processes engine-level controls, then finalizes per-frame input state.
2. Update updates the window and SceneManager.
3. Render calls BeginFrame, Present, EndFrame on CIRenderDevice when initialized.
4. UpdateFPS applies the optional frame limiter. In the current source m_bLimitFPS defaults false.

Window resize callbacks publish a resize event; the engine subscription calls CIRenderDevice::Resize. Vulkan uses resize/out-of-date handling to recreate swapchain-dependent resources.

### Vulkan initialization route

5. CVulkanRenderDevice creates CVulkanDevice and asks it to create the VkInstance.
6. It creates a GLFW Vulkan surface, chooses a physical device and graphics/present queue families, then creates the logical device and command pool.
7. It creates CVulkanSwapchain and frame resources, then initializes texture and pipeline managers and loads configured actors.
8. CVulkanRenderer creates per-frame command buffers, synchronization objects, the skin palette manager, and frame descriptor resources.
9. The command list is connected to the renderer and the device marks itself initialized.

### One Vulkan frame

| Phase | What the implementation does | Source |
|---|---|---|
| BeginFrame | Waits for this frame slot's fence, acquires a swapchain image, begins that slot's command buffer. Out-of-date acquisition triggers resize handling. | VulkanRenderer::BeginFrame |
| Render pass | Begins the swapchain render pass, sets viewport/scissor, and clears color/depth. | VulkanRenderer::BeginSwapchainRenderPass |
| Gather draws | Calls SceneManager::Render, which asks components for render instances; backend sorts and binds the batch state. | VulkanRenderDevice::Present; VulkanRenderer::FlushRenderItems |
| Submit/present | Ends command recording, submits to graphics queue with semaphores/fence, then calls vkQueuePresentKHR. | VulkanRenderer::EndFrame |
| Advance | Clears render queue and advances the frame-in-flight slot modulo MAX_FRAMES_IN_FLIGHT (currently 2). | VulkanRenderer::EndFrame; RenderDevice.h |

**Naming note** CIRenderDevice::Present does not itself call vkQueuePresentKHR in the Vulkan backend. CVulkanRenderDevice::Present updates buffers, asks the scene to submit/flush items; CVulkanRenderer::EndFrame makes the actual queue-present call.

The current code uses two different indices: current frame-in-flight slot selects per-frame command buffers, fences, semaphores and uniform buffers; acquired swapchain image index selects the framebuffer/image to render to and is used for image-associated synchronization. They are related by acquisition but are not interchangeable.

# 5. Scene, model, and animation flow

## Initial scene

SceneManager::Initialize allocates a CScene and looks up actor info named Warrior_Male. If it has an asset, SpawnActorEntity creates an entity, adds a transform and skeletal mesh component, sets a position/scale/rotation, assigns the skeletal actor asset and initializes the component. This is a sample scene path rather than a general gameplay framework.

## Animation update

SceneManager::Update iterates scene entities and updates their skeletal mesh components. The sample toggles animation updates using keys 1 and 2. CSkeletalMeshComponent owns a CAnimator, connects it to the loaded skeleton and animation library, and starts the WarriorMale/OnehandSword/Idle animation when initialized.

## Render-item path

10. SceneManager::Render visits entities and asks each skeletal component to BuildRenderItemsNew.
11. The component reads the entity world transform, uploads current final bone matrices into CSkinPaletteManager, and creates one SRenderInstance per mesh batch (plus applicable attachments).
12. SceneManager submits those instances to CRenderQueue. The Vulkan renderer sorts them, binds pipeline, vertex/index buffers, material and frame/bone descriptor sets, pushes per-object data, then calls DrawIndexed.
13. At the end of a frame the render queue is cleared.

| Concept | Role |
|---|---|
| Actor asset | A named, configured character or object asset; actor config records paths and import/animation metadata. |
| Model / mesh | Imported geometry and material/batch data used by the renderer. |
| Entity / component | Scene instance and attached behavior/data. Transform and skeletal mesh are the sample components used here. |
| Render instance | Per-instance data such as batch pointer, model matrix, skin-palette range, flags and sort key. |
| Render queue | Temporary collection of render instances, sorted before backend submission. |

# 6. Renderer architecture

## Shared contract

CIRenderDevice defines backend-neutral operations for initialization, frame control, buffers, textures, shaders, pipelines, materials, buffer updates, command lists and resize. Systems should prefer this interface unless they truly need a backend-specific feature. MAX_FRAMES_IN_FLIGHT is defined in RenderDevice.h and currently equals two.

## Vulkan building blocks

| Type | Responsibility / reading focus |
|---|---|
| CVulkanDevice | Instance, surface, physical/logical device, queues, command pool, memory/format helpers. |
| CVulkanSwapchain | Swapchain images, image views, render pass, depth/framebuffer resources and swapchain lifetime. |
| CVulkanRenderer | Frame lifecycle, command buffers, sync, frame descriptors, camera UBO update, render-item flush. |
| CVulkanCommandList | Records explicit bind/draw operations against the current command buffer. |
| CVulkanPipeline / shader | Creates Vulkan pipeline state and shader modules from SPIR-V paths resolved for the backend. |
| DescriptorContext / DescriptorSet | Allocates and updates descriptor resources for material, frame and bone data. |
| VulkanBuffer / VulkanTexture2D | Own backend resource handles, allocation and upload behavior; trace destroy path too. |
| CVulkanSyncObject | Semaphores and fences coordinating frame slots and swapchain image usage. |

### Pipeline and shader contract

CPipelinesManager initializes default static-mesh and skeletal-mesh pipelines. It resolves backend-specific shader paths, defines vertex input/topology/culling/front-face/depth state, and for Vulkan supplies descriptor layouts for material (set 0), frame (set 1), and bones (set 2). The Vulkan pipeline layout and shader declarations must agree with these bindings and the data written by renderer/material/skin palette code.

### OpenGL path

OpenGL has parallel device/resource/renderer abstractions. The window manager requests an OpenGL 4.6 core context and OpenGL backend resource/shader classes live under Renderer/OpenGL. The generic flow remains similar, but backend resource operations differ. The current Sandbox entry initializes OpenGL; F1 requests Vulkan and F2 requests OpenGL via SwitchGraphicsAPI.

## 7. Core services and math

| Area | What it contributes | Where to start |
|---|---|---|
| Window | GLFW lifecycle, window mode, callbacks, camera access, resize/close signals. | Core/src/Window/WindowManager.cpp |
| Input and events | Callbacks become typed events and update frame-based keyboard/mouse state. | Core/include/Events; Core/src/Input |
| Timer | Delta-time updates and optional frame limiting. | Core/include/Time/TimerManager.h |
| Logging/config | Log/error output and JSON configuration access. | Core/include/Logging; Core/include/Config |
| Memory manager | Tagged tracked allocation helpers, leak reporting and allocation diagnostics. | Core/include/Memory/MemoryManager.h; MemoryTags.h |
| Math | Custom vectors/matrices/quaternions, transforms and geometry helpers used by scene/camera/model code. | Math/include/EngineMath.h; Transform.h |

The memory macros route allocations through CMemoryManager, often with a memory tag. Their use does not remove the need to understand ownership: pair creation and cleanup, and inspect whether a resource is owned directly, by a manager, or through a smart pointer.

## 8. Assets and configuration

| Config | Likely consumer / what to inspect |
|---|---|
| Assets/Config/EngineConfig.json | CConfigManager; global runtime settings. |
| Assets/Config/EngineActors.json | CActorsManager; named model files, CRC, UV flip, animation profile and attachment data. |
| Assets/Config/EngineShaders.json | Shader manager/path resolution helpers; correlate names with shaders/ files. |
| Assets/Config/EngineTextures.json | CTexturesManager; texture descriptions/paths. |
| Assets/Animations/ | Animation-profile JSON consumed through actor/model importer flow. |
| shaders/vulkan/ | GLSL source, compiled SPIR-V files, and compile.bat helper. |

Several asset constants in CoreDefines.h are relative to the working directory, for example Assets/Config/EngineActors.json and Assets/Logs/EngineLog.txt. Sandbox CMake sets the debugger working directory to the repository root. If an asset mysteriously fails to load, check the working directory and configured path first.

ActorsManager parses actor JSON and loads configured actors through the importer. The initial scene currently requests the exact key Warrior_Male. The asset tree also contains many model/texture files; avoid inferring from a file's presence that it is referenced by the active configuration.

# 9. Build and runtime notes

The project uses CMake and C++20. The root options enable the Sandbox by default and disable the Editor by default. Engine is a static library linked to subsystem libraries and external dependencies. README describes expected Vulkan SDK and third-party packages; inspect actual CMake paths and local libraries before assuming a clean machine can build it.

```
cmake -S . -B build
cmake --build build --config Debug
```

Build examples only: exact success depends on the compiler, Vulkan SDK, and prebuilt libraries present on the machine. The README mentions dependencies under Extern/Include and Extern/lib, while source tree naming and case may vary; verify paths in CMakeLists.txt for the current platform.

| Runtime control | Current behavior in source |
|---|---|
| F11 | Toggle windowed/fullscreen mode. |
| F1 | Switch graphics API to Vulkan. |
| F2 | Switch graphics API to OpenGL. |
| Escape | Request engine shutdown. |
| 1 / 2 | In scene update, pause/resume the sample skeletal animator. |

**First run** The sandbox starts with OpenGL. To begin in Vulkan without pressing F1, change the API argument in Sandbox/Main.cpp. Switching backend recreates the window and render device, then destroys/reinitializes the scene.

# 10. Known code facts to recheck

These are useful orientation clues and likely troubleshooting entry points. They are not judgments about intended final design.

- Sandbox/Main.cpp currently initializes API_OPENGL even though Vulkan is implemented and F1 switches to it.
- CVulkanDevice targets VK_API_VERSION_1_4 and enables VK_LAYER_KHRONOS_validation during instance creation. Confirm the SDK/runtime and validation layer available on the machine you use.
- CVulkanDevice::IsDeviceSuitable currently requires a discrete GPU, geometryShader support, complete graphics/present queue families, and VK_KHR_swapchain. Integrated GPUs may be rejected.
- CAnubisEngine::Initialize does not check every subsystem Initialize return value before proceeding; when debugging failed startup, inspect each returned bool and logs.
- CVulkanRenderDevice::Shutdown assumes Vulkan subobjects were created. Partial initialization failure paths are worth reviewing if initialization can fail midway.
- CMake configuration contains platform-specific dependency paths and should be checked against the actual machine/environment before changing build scripts.
- The repo has OpenGL and Vulkan shader variants. A shader source edit does not automatically mean the Vulkan SPIR-V was regenerated; check compile.bat and the pipeline path.

# 11. Study plan and self-quiz

Use one 10–20 minute session at a time. First answer from memory, then inspect only the named files, then correct yourself in a few sentences. No weekly schedule is required; restarting after a missed week is fine.

| Session | Read | Try to recall |
|---|---|---|
| 1. Startup | Sandbox/Main.cpp; AnubisEngine::Initialize | Who constructs what? In what order are services and renderer initialized? |
| 2. Tick and input | AnubisEngine::Tick / HandleInput; WindowManager callbacks | When are OS events polled relative to input state update? |
| 3. Vulkan frame | VulkanRenderDevice and VulkanRenderer Begin/Present/EndFrame | What do frame slot and swapchain image indices refer to? Where is actual queue present? |
| 4. A model appears | ActorsManager; SceneManager; SkeletalMeshComponent | How does config become an entity component and |

| | | |
|---|---|---|
| | | then render batches? |
| 5. One draw | RenderQueue; VulkanRenderer::FlushRenderItems; VulkanCommandList | Which state/resources are bound before the indexed draw? |
| 6. Resource lifetime | VulkanBuffer or VulkanTexture2D; VulkanRenderDevice destroy methods | Who creates, owns, updates, and destroys this resource? |
| 7. Shader pipeline | EngineShaders.json; CoreUtils shader resolver; PipelinesManager; VulkanPipeline | How are source/SPIR-V paths and descriptor layouts connected? |
| 8. Resize/shutdown | WindowManager resize events; VulkanSwapchain; Shutdown methods | Which objects depend on swapchain/device lifetime and what must be recreated? |

### Quick recall questions

14. Why is the render device an interface rather than a concrete Vulkan device throughout the engine?
15. What is the difference between a mesh batch and a render instance?
16. Where is the camera view/projection data updated for Vulkan?
17. How does a window resize reach the Vulkan swapchain code?
18. What data does a skeletal render instance need in addition to its model matrix?
19. Which paths are relative to the working directory?
20. If the warrior does not appear, what are the first three files/configs you would inspect?

### Reusable session note

| Prompt | Your notes |
|---|---|
| Date | |
| Area reviewed | |
| What I remembered before reading | |
| What the code actually does | |
| Question / next file | |

## 12. Glossary and source index

| Term | Meaning here |
|---|---|
| Backend | The concrete graphics implementation selected behind CIRenderDevice: OpenGL or Vulkan. |
| Swapchain | Vulkan images presented to the window system; image acquisition/presentation is distinct from CPU frame slots. |
| Frame in flight | A CPU/GPU frame slot whose resources can be reused after synchronization; currently two slots. |
| Command buffer | Recorded Vulkan commands submitted to a queue. |
| Descriptor set | Vulkan bindings that make buffers/images available to shaders. |
| Pipeline | GPU state/shader configuration used for draws; includes vertex input, topology, raster/depth state and layout. |
| Skin palette | A collection/range of bone matrices made available to skeletal shaders. |
| Service locator | Registry that lets systems retrieve shared manager/service instances. |

### Source index by question

| If you want to understand… | Start here |
|---|---|
| Main loop / API switch | Engine/AnubisEngine.cpp; Sandbox/Main.cpp |
| Window creation and callbacks | Engine/Core/src/Window/WindowManager.cpp; Events/ |
| Backend-neutral render objects | Engine/Core/include/API/ |
| Vulkan device selection | Engine/Renderer/src/Vulkan/VulkanDevice.cpp |
| Swapchain | Engine/Renderer/src/Vulkan/VulkanSwapchain.cpp; VulkanRenderDevice.cpp |
| Frame synchronization / draw | Engine/Renderer/src/Vulkan/VulkanRenderer.cpp; VulkanSyncObject.cpp; VulkanCommandList.cpp |
| Static/skeletal pipelines | Engine/Renderer/src/Device/PipelinesManager.cpp; VulkanPipeline.cpp |

| | |
|---|---|
| Actor import | Engine/Model/src/Services/ActorsManager.cpp; AssimpModelImporter.cpp |
| Animation | Engine/Model/src/Model/Animator.cpp; Animation.cpp; Scene/src/Entity/SkeletalMeshComponent.cpp |
| Scene traversal | Engine/Scene/src/Core/SceneManager.cpp; Scene.cpp |
| Memory tracking | Engine/Core/src/Memory/MemoryManager.cpp; include/Memory/ |

## A practical return-to-code sequence

When you have your development machine again, begin with one normal clean build and one launch in the backend you want to study. Record the compiler/configuration and any first error. Then choose one vertical slice such as warrior animation to indexed draw, and trace it without refactoring. Update this guide's re-entry notes when your understanding or the code changes.

You do not need to keep every implementation detail in memory. The goal is to remember the shape of the system and know where to look next.